

Operating Systems Exercise 1

Threading & Synchronization

Part A - Theory (submit as a separate PDF)

Submit answers in a single PDF file.

Important: Use **clear, numbered headers** for each question (e.g. "TQ1", "TQ2", etc.) so the grader can locate your answers easily. Submissions with unclear or missing headers may be mis-graded.

TQ1 — Concurrency vs. parallelism

A program has 3 threads that each need 40_{ms} of CPU time. There is no I/O and no blocking.

1. On a single core, assuming the system switches between runnable threads and context-switch cost is negligible, what is the minimum wall-clock time until all threads finish?
2. On a 3-core CPU, same assumptions, what is the minimum wall-clock time?
3. If each context switch costs 0.2_{ms} , and on the single core the system forces a context switch after every 5_{ms} of CPU execution of the currently running thread, what is the extra overhead added purely due to context switching until completion?

Assume a context switch also occurs after the final 5_{ms} chunk that completes the last thread.

Solution

1. **Single core:** only one thread runs at a time.

Total CPU work = $(3 \cdot 40 = 120)_{ms}$. With zero overhead, wall-clock = 120_{ms} .

There's only **concurrency**, no parallel execution.

2. **Three cores:** all 3 threads can run simultaneously. Each needs 40_{ms} CPU, so in the best scenario wall-clock = 40_{ms} (perfect parallelism).

3. With forced switching every 5_{ms} on a single core:

- Each thread needs 40_{ms} , so $\frac{40}{5} = 8$ chunks per thread (i.e. 8 turns to finish).
- Total chunks executed for all threads = $(3 \cdot 8 = 24)$.
- Each chunk ends with a context switch (including after the final chunk by assumption), so total context switches = 24.
- Overhead = $24 \cdot 0.2 = 4.8_{ms}$

TQ2 — Interleavings

Question.

Shared/global integer x starts at 0. Two threads, A and B, run concurrently the following code **once each**:

```
t = x
t = t + 1
x = t
```

Assume t is a **thread-local** variable (each thread has its own t).

1. What final values of x are possible?
2. Give an explicit interleaving run that leads to each possible final value you stated in (1).

Solution

1. Possible final values: **1 or 2**.

2. Interleavings:

- $x = 2$:
 - A: $tA = x$ ($tA=0$)
 - A: $tA = tA+1$ ($tA=1$)
 - A: $x = tA$ ($x=1$)
 - B: $tB = x$ ($tB=1$)
 - B: $tB = tB+1$ ($tB=2$)
 - B: $x = tB$ ($x=2$)
- $x = 1$:
 - A: $tA = x$ ($tA=0$)
 - B: $tB = x$ ($tB=0$) $\leftarrow$ both read 0
 - A: $tA = tA+1$ ($tA=1$)
 - B: $tB = tB+1$ ($tB=1$)
 - A: $x = tA$ ($x=1$)
 - B: $x = tB$ ($x=1$) $\leftarrow$ overwrites with 1 again

TQ3 — TAS vs Ticket lock

Consider the following two locks:

- a **test-and-set (TAS) spinlock**
 - a **ticket lock** as we learned in the lecture (each thread gets `my_ticket = fetch_add(ticket,1)` and waits for `cur_ticket == my_ticket`).
1. Which lock guarantees **FIFO acquisition order**? Prove it.
Hint: Identify a per-thread “position” value assigned on arrival (e.g., a ticket number), and argue that the lock can be acquired only in increasing order of that value.
 2. Which lock guarantees **bounded waiting** (no starvation), assuming every thread that acquires the lock eventually releases it? Prove it.
Hint: If a thread gets ticket k and the current ticket is c , how many threads can enter before it? Express this as a bound.
 3. Give a short argument (one paragraph) why TAS can starve a thread.

Solution

1. **FIFO acquisition order:** ticket lock.
 - Each arriving thread atomically gets a unique ticket number k (strictly increasing).
 - The lock grants entry exactly when `cur_ticket == k` .
 - `cur_ticket` increases by exactly 1 on each release.
 - Therefore, tickets are served in increasing order $\Rightarrow$ **FIFO**.

2. Bounded waiting: ticket lock.

Suppose a thread gets ticket k . Let `cur_ticket` at that moment be c .

- Exactly $(k - c)$ tickets are ahead of it.
- Each release advances `cur_ticket` by 1.
- Assuming each holder eventually releases, the thread will enter after at most $(k - c)$ releases.
- Thus waiting is bounded $\Rightarrow$ **no starvation**.

3. Why TAS can starve:

TAS lock acquisition is a "race" on every release: when the holder releases the lock, all contenders repeatedly try the test-and-set operation. A thread that repeatedly loses due to unlucky timing or scheduling can keep losing forever, because there is no queue and no ordering rule that guarantees it will eventually win.

TQ4 — Producer/Consumer

We have a bounded buffer (queue) of capacity `CAP`, shared by multiple producers and consumers. Bounded semaphore value is always in $[0, CAP]$; unbounded semaphore has no fixed upper bound.

We use three synchronization objects:

- `mutex` — mutual exclusion for the buffer
- `empty` — a **bounded semaphore** with value always in $[0, CAP]$, initialized to `CAP` (number of empty slots)
- `full` — a **bounded semaphore** with value always in $[0, CAP]$, initialized to `0` (number of full slots)

A student wrote the following high-level algorithm:

Producer (buggy version):

1. `acquire mutex`
2. `wait(empty)`
3. `insert one item into the buffer`
4. `signal(full)`
5. `release mutex`

Consumer (buggy version):

1. `acquire mutex`
2. `wait(full)`
3. `remove one item from the buffer`
4. `signal(empty)`
5. `release mutex`

1. Explain why this algorithm can deadlock. Your explanation must include a concrete execution scenario (who holds what and who is waiting for what).
2. Give the **minimal fix** (change as little as possible) to make the algorithm deadlock-free, and explain why your fix works.

Solution

1. Why it can deadlock (example scenario):

Consider the buffer is initially empty:

- `full = 0`, `empty = CAP`.

Now a consumer runs:

- Consumer acquires `mutex` .
- Consumer calls `wait(full)` . Since `full = 0` , the consumer blocks **while still holding** `mutex` .

Now a producer runs:

- Producer tries to acquire `mutex` , but it is held by the blocked consumer, so the producer blocks.

We now have a deadlock:

- The consumer is waiting for `full > 0` , which can only happen if a producer inserts an item and signals `full` .
- But the producer cannot insert an item because it cannot acquire `mutex` .
- The consumer will not release `mutex` because it is blocked inside `wait(full)` .

This is circular waiting caused by blocking on a semaphore while holding `mutex` .

2. Minimal fix:

Move the `wait(...)` operations **before** acquiring `mutex` (so no thread blocks while holding `mutex`).

Fixed Producer:

1. `wait(empty)`
2. `acquire mutex`
3. `insert one item into the buffer`
4. `release mutex`
5. `signal(full)`

Fixed Consumer:

1. `wait(full)`
2. `acquire mutex`
3. `remove one item from the buffer`
4. `release mutex`
5. `signal(empty)`

Why it works:

- A thread may block in `wait(empty)` / `wait(full)` , but it blocks **without holding** `mutex` .
- Therefore it cannot prevent the other side from acquiring `mutex` to make progress and eventually signal the semaphore.
- This removes the circular-wait condition, so the system is deadlock-free (assuming threads that acquire `mutex` eventually release it).